Algorithmes et Pensée Computationnelle

Structures de données, Itération et récursivité - Basiques

Le but de cette séance est de mettre en pratique les connaissances acquises en cours sur les structures de données (tuples, listes et dictionnaires), l'itération et la récursivité. À l'issue de cette séance, vous serez capables de distinguer les structures de données muables et immuables, ainsi que d'écrire des programmes faisant appel à l'itération et à la récursivité.

Le temps mentionné (🕓) est à titre indicatif.

Pour les exercices où vous devez déterminer ce qu'affiche un programme, résolvez-les d'abord à la main avant d'utiliser IntelliJ, afin de simuler les conditions d'examen.

1 Structures de données

1.1 Tuples

Pour rappel, les tuples en Python sont des séquences immuables, ce qui signifie qu'une fois créés, leurs éléments ne peuvent être ni ajoutés, ni supprimés, ni remplacés. L'une des particularités des tuples est la possibilité d'y inclure des données de types différents.

En Python, pour créer un tuple, il suffit de définir une variable et de lui affecter des valeurs séparées par des virgules, généralement placées entre parenthèses :

Exemple

```
1  mon_tuple = (1,"une chaîne de caractères",3,4)
```

Question 1: (🕓 *5 minutes*) [Python] **Manipulation d'un tuple**
Créez un tuple nommé `mon_tuple` contenant les chiffres 1, 2, 3, 4 et 5. Affichez le quatrième élément de ce tuple. Obtenez le nombre d'éléments contenus dans votre tuple et stockez le résultat dans une nouvelle variable nommée `taille_tuple`. Pour finir, affichez le contenu de `taille_tuple`.

1.2 Listes

Contrairement aux tuples, les listes sont muables : leurs éléments peuvent être ajoutés, supprimés ou remplacés.

Pour créer une liste, il suffit de définir une variable avec des valeurs entre crochets :

Exemple

```
1  ma_liste = [1,2,3,4,5]
```

Comme pour les tuples, on accède aux éléments d'une liste avec des crochets `[]`, en utilisant des indices qui commencent à zéro. La fonction `len()` permet également d'obtenir le nombre d'éléments de la liste :

Exemple

```
1  ma_liste = [1,2,3,4,5]
2  print(ma_liste[1])    # affiche 2 (le deuxième élément)
3  print(len(ma_liste)) # affiche 5 (le nombre d'éléments)
4
```

Question 2: (*5 minutes*) [Python] **Manipulation d'une liste**
Qu'affiche le code suivant ?

```
1 ma_liste = [1,2,2,4,5]
2 ma_liste[2] = 3
3 ma_liste.append(6)
4 ma_liste.insert(0,0)
5 print(ma_liste)
```

Choisissez parmi les éléments suivants :

(A) `[1, 2, 2, 4, 5, 6]`
(B) `[0, 1, 2, 3, 4, 5]`
(C) `[0, 1, 2, 3, 4, 5, 6]`
(D) `[0, 1, 3, 2, 4, 5, 6]`

1.3 Itération

Quelques rappels de concepts théoriques :

- L'itération désigne l'action de répéter un processus (généralement à l'aide d'une boucle) jusqu'à ce qu'une condition particulière soit remplie.
- Il y a deux types de boucles qui sont majoritairement utilisées :

  Boucle `for` — Python Une boucle `for` parcourt un itérable (liste, tuple, chaîne de caractères, dictionnaire, etc.). Syntaxe : `for element in iterable:`. À chaque itération, `element` prend successivement la valeur de chaque élément de `iterable`.

  Boucle `for` — Java La syntaxe est la suivante : `for (type nomDeVariable; condition de fin de boucle; incrémentation à chaque itération)` suivie d'une accolade. Il est également possible d'utiliser la syntaxe suivante pour itérer directement sur les éléments d'une liste : `for(var nomDeVariable: nomDeLaListeOuTuple)` ou `for(var nomDeVariable: nomDuDictionnaire.keySet())` pour un dictionnaire. Cette notation est appelée `for each`.

  Boucle `while` Une boucle `while` répète un bloc d'instructions *tant que* sa condition est vraie.
  - Python : `while condition:`, suivi d'un bloc indenté.
  - Java : `while (condition) { ... }`, avec la condition entre parenthèses et le bloc délimité par des accolades.
- La fonction `range(n)` permet de créer un ensemble de nombres de 0 à la valeur passée en argument moins 1 `(n-1)`. Lorsqu'elle est combinée à une boucle `for`, on peut itérer sur une liste de nombres de 0 à `n-1`.

- Si vous avez fait une erreur dans votre code, il se peut que la boucle `while` ne remplisse jamais la condition lui permettant de sortir. On parle dans ce cas d'une `boucle infinie`. Ceci peut entraîner un crash de votre programme, voire de votre ordinateur. Il faut toujours s'assurer qu'il y ait une condition valide dans une boucle `while`.

Question 3: (*5 minutes*) [Python] **Boucle for**
Qu'affiche le code suivant ?

```
1 ma_liste = [1,2,3,4,5]
2 for c in ma_liste:
3     if 2*c in ma_liste:
4         print(c)
5
```

Choisissez parmi les possibilités suivantes :

(A)	(B)	(C)	(D)
1 2	2 4	1 2 3 4 5	3 4 5

Question 4: (*5 minutes*) [Java] **Boucle for**
Qu'affiche le code suivant ?

```
List maListe = List.of(2,3,4,5,6);
for(int i=0;i<maListe.size();i++){
    System.out.print(maListe.get(i));
}

```

Choisissez parmi les possibilités suivantes :

(A) `5 4 3 2 1`

(B) `1 2 3 4 5`

(C) `4 3 5 1 2`

(D) `2 3 4 5 6`

Question 5: (*5 minutes*) [Python] **Boucle while**
Qu'affiche le code suivant ?

```
i = 0
result = ""
while i<5:
    result = result + str(i)
    i += 1
print(result)

```

Choisissez parmi les possibilités suivantes :

(A) `0 1 2 3 4`

(B) `1 2 3 4 5`

(C) `4 3 2 1 5`

(D) `2 3 4 0 1`

Question 6: (*5 minutes*) [Java] **Boucle while**
Qu'affiche le code suivant ?

```
int i = 0;
while(i<5){
    i++;
    System.out.print(i + " ");
}

```

Choisissez parmi les possibilités suivantes :

(A) `0 1 2 3 4`

(B) `1 2 3 4 5`

(C) `4 3 2 1 0`

(D) `5 4 3 2 1`

Question 7: (🕓 *5 minutes*) [Python] **Boucle for et fonction range()**
Qu'affiche le code suivant ?

```
ma_liste = ["J'aime","Python"]
for x in range(len(ma_liste)):
    print(x)
    print(ma_liste[x])
```

Choisissez parmi les possibilités suivantes :

(A)	(B)	(C)	(D)
0 J'aime 1 Python	J'aime 1 Python 2	J'aime 0 Python 1	1 J'aime 2 Python

Question 8: (🕓 *5 minutes*) [Python] **Boucle for et fonction range()**
En utilisant la fonction `range()` et une boucle `for`, calculez la somme des entiers de 0 à 20 et affichez le résultat.

Question 9: (🕓 *10 minutes*) [Python] **Boucle while et input()**
À l'aide d'une boucle `while`, demandez à l'utilisateur de rentrer une valeur. Tant que cette valeur ne correspond pas à 10, le programme redemande à l'utilisateur une nouvelle valeur.

Question 10: (🕓 *5 minutes*) [Python] **Listes en compréhension**
Créez la liste `nombres` contenant les entiers de 0 à 10 inclus. Par compréhension de liste, stockez ses entiers pairs dans `nombres_pairs`, puis affichez cette liste.

Question 11: (🕓 *10 minutes*) [Java] **Itérateurs**
En Java, définissez une liste `nombres` contenant l'ensemble des nombres compris entre 0 et 10. En utilisant un `Iterator`, supprimez tous les nombres impairs présents dans `nombres`. Affichez le contenu de `nombres`.

1.4 Dictionnaires

Les dictionnaires sont des listes associatives, c'est-à-dire des listes qui lient une valeur à une autre. Dans un dictionnaire en papier, les mots sont liés à leur définition.

Dans un dictionnaire Python, les éléments sont liés par une relation dite `clé, valeur`. La clé est le moyen de "retrouver" notre valeur dans notre dictionnaire. Par exemple, dans un dictionnaire en papier, nous pouvons retrouver une définition en cherchant le mot qui lui correspond. Alternativement, nous pouvons retrouver le contenu d'une page d'un livre en utilisant le numéro de celle-ci. Dans ce cas, le numéro est la clé et le texte de la page, la valeur.

Pour créer un dictionnaire, il suffit de lister les couples `clé, valeurs` entre deux accolades :

```
elon_musk = {
    "prénom": "Elon",
    "nom": "Musk",
    "age": 48,
    "talents": ["programmation", "entrepreneuriat", "aéronautique"]
}

```

Question 12: (🕓 *5 minutes*) [Python] **Manipulation d'un dictionnaire**
Créez un dictionnaire nommé `fr_eng` contenant les associations suivantes :

```
{
    "chat": "cat",
    "chien": "dog",
```

```
    "oiseau": "bird",
    "poule": "chicken",
    "papillon": "butterfly",
    "souris": "mouse",
    "ours": "bear",
    "mouton": "sheep",
    "cochon": "pig"
}
```

Ce dictionnaire contient des mots en français (`clés`) associés à leur traduction en anglais (`valeurs`).

Accédez à la traduction du mot “souris”, et stockez le résultat dans une variable nommée `souris_traduite`, puis calculez le nombre d’éléments contenus dans le dictionnaire et stockez le résultat dans une variable nommée `taille_fr_eng`. Affichez le contenu des deux variables que vous venez de créer.

Question 13: (🕓 *5 minutes*) [Python] **Manipulation d’un dictionnaire**
Qu’affiche le programme suivant ?

```
elon_musk = {
    "prenom": "Elon",
    "nom": "Musk",
    "age": 48,
    "talents": ["programmation", "entreprenariat", "aeronautique"]
}

print(elon_musk["talents"][2])
print("aeronautique" in elon_musk["talents"])
```

Choisissez parmi les possibilités suivantes :

(A) `programmation`
`True`

(B) `Musk`
`False`

(C) `48`
`False`

(D) `aéronautique`
`True`

Question 14: (🕓 *5 minutes*) [Python] **Manipulation d’un dictionnaire**

1. Vous disposez de deux dictionnaires contenant les notes d’étudiants dans différentes matières : notes_biologie et notes_chimie. Chaque dictionnaire associe le nom d’un étudiant à sa note dans la matière correspondante. Votre objectif est de créer un nouveau dictionnaire qui regroupe les notes de chaque étudiant sous forme de liste, où chaque liste contient d’abord la note de biologie puis la note de chimie. Complétez le programme suivant :

```
if __name__ == '__main__':
    notes_biologie = {"Charlie": 2, "Alice": 6, "Bob": 5.5}
    notes_chimie = {"Charlie": 4, "Alice": 5, "Bob": 6}
    # Votre code ici:

    # Resultat:
    # {"Charlie": [2, 4], "Alice": [6, 5], "Bob": [5.5, 6]}
```

2. Affichez le nombre d’étudiants dont la note moyenne est plus grande que 5.

1.5 Structures de données en Java

Dans cette partie, les exercices devront être effectués en Java.
Les syntaxes des principales structures de données abordées dans cette partie sont les suivantes :
Déclaration et affectation d'un tuple (array) en Java de type Object[] :

```
Object[] monTuple = {1,2,3,4};
```

Déclaration et affectation d'un tuple (array) en Java de type String[] :

```
String[] monTuple = {"UNIL","ACT"};
```

En général, pour la déclaration, on écrit : `T[] nomVariable`, où `T` est le type d'éléments dans le tuple.

Déclaration et affectation d'une liste en Java :

```
List maListe = List.of(1,2,3,4);
```

Déclaration et affectation d'un dictionnaire en Java :

```
HashMap monDictionnaire = new HashMap(Map.of("un", 1, "deux", 2));
```

Informations utiles

- En Java, les listes sont immuables, comme les tuples en Python. Au cas où vous devriez modifier une liste, on vous recommande d'utiliser une liste liée (`LinkedList`). Une liste liée se déclare comme suit :

```
List liste = List.of(1,2,3,4);
LinkedList maListe = new LinkedList(liste);
```

- En Java, les listes ne peuvent contenir que des éléments de même type.

Question 15: (*10 minutes*) [Java] **Manipulation des listes**
Créez une liste nommée `maListe` contenant les nombres 1, 2, 3, 4 et 5. Affichez le deuxième élément de la liste ainsi que la taille de la liste.

Créez une liste `maListeM` liée à la liste `maListe`. Ajoutez le chiffre 6 à la fin de la liste, et le chiffre 0 au début de cette dernière.

Ajoutez ceci au début de votre code :

```
import java.util.List;
import java.util.LinkedList;
```

Ajoutez ceci à la fin de votre code :

```
for(int i=0;i<maListeM.size();i++){
    System.out.println(maListeM.get(i));
}
```

Question 16: (*15 minutes*) [Java] **Manipulation de dictionnaires**
Créez un dictionnaire `monDictionnaire` contenant les associations suivantes :
`("étudiants", 14000, "enseignants", 2300, "collaborateurs", 0)`

Affichez le nombre d'étudiants. Obtenez la taille du dictionnaire et stockez le résultat dans une variable `tailleDictionnaire`. Affichez le contenu de `tailleDictionnaire`. Modifiez la valeur de `collaborateurs`, remplacez 0 par 950. Rajoutez un nouvel élément `pays` dans votre dictionnaire avec pour valeur 86.

Ajoutez ceci au début de votre code :

```
import java.util.HashMap;
import java.util.Map;
```

Ajoutez ceci à la fin de votre code :

```
for (String key: monDictionnaire.keySet()){
    System.out.println(key + ": "+ monDictionnaire.get(key));
}
```

2 Récursivité

La récursivité consiste à faire en sorte qu'une fonction s'appelle elle-même, directement ou indirectement.

Question 17: (*5 minutes*) [Python] **Lecture de code**
Qu'affiche le programme suivant ?

```
def fonction_p(x) :
    print(x)
    if x == 0 :
        pass
    else :
        fonction_p(x-1)

fonction_p(5)
```

Choisissez parmi les possibilités suivantes (on écrit tout sur la même ligne par souci de concision) :

(A) `4 3 2 1 0`
(B) `5 4 3 2 1 0`
(C) `0 1 2 3 4`
(D) `1 2 3 4 5 0`

Question 18: (*5 minutes*) [Python] **Lecture de code**
Qu'affiche le programme suivant ?

```
def fonction_p(x) :
    if x == 0 :
        pass
    else :
        fonction_p(x-1)
        print(x)

fonction_p(5)
```

Choisissez parmi les possibilités suivantes (on écrit tout sur la même ligne par souci de concision) :

(A) `4 3 2 1 0`
(B) `5 4 3 2 1`
(C) `0 1 2 3 4`
(D) `1 2 3 4 5`

Question 19: (*10 minutes*) [Java] **Méthode getBinaryRepresentation()**
Le calcul de la représentation binaire d'un entier plus grand que 0 peut être résolu de manière récursive :

$$f(a) = \begin{cases} f\left(\left\lfloor \frac{a}{2} \right\rfloor\right) + \texttt{"0"} & \text{si } a \bmod 2 = 0 \\ f\left(\left\lfloor \frac{a}{2} \right\rfloor\right) + \texttt{"1"} & \text{si } a \bmod 2 = 1 \end{cases} \tag{1}$$

La notation $\left\lfloor \frac{a}{2} \right\rfloor$ désigne une division avec arrondi inférieur. Le cas de base est le suivant :

$$f(0) = \texttt{""} (\text{la chaîne vide}) \tag{2}$$

Ici, $f(a)$ renvoie la représentation binaire sous forme d'une chaîne de caractères du nombre entier $\mathbf{a} \geq \mathbf{1}$ et le symbole $+$ fait une concaténation de chaînes de caractères.

Écrivez la méthode `getBinaryRepresentation(String a)` équivalente à f (à part le fait que l'entrée soit une chaîne de caractères) suivant une approche récursive.